Beágyazottszoftver-fejlesztés 2025. 09. 18-25.

Szabványosítás - CMSIS

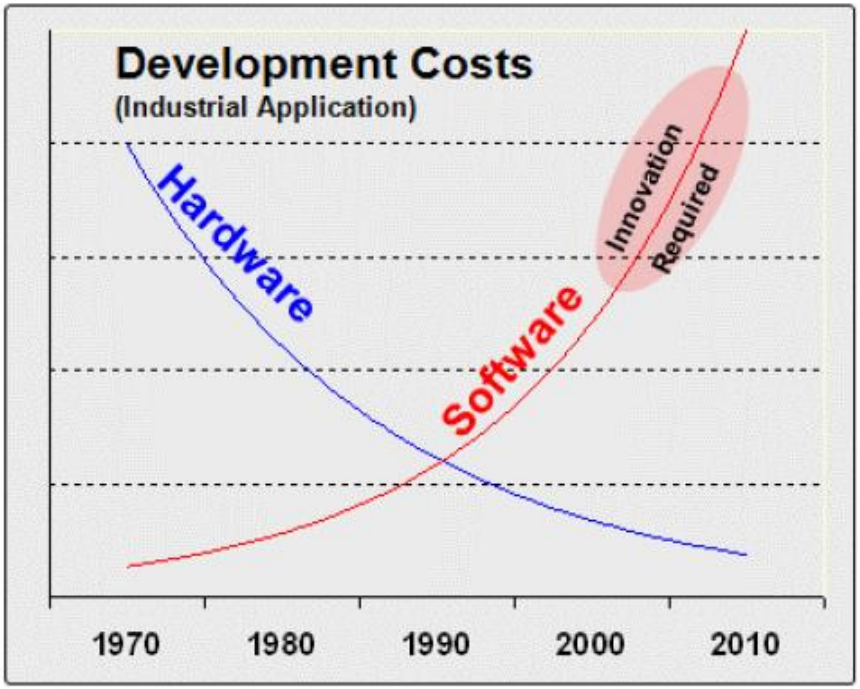


Mesterséges Intelligencia és
Rendszertervezés
Tanszék

© BME-MIT

Szabványosítás célja

- Szoftverek: egyre bonyolultabb és komplexebb rendszerek
- Fejlesztési idő egyre nő <-> Piacra kerülési idő fontos
- Rengeteg különböző alkalmazás, amik hasonló mikrokontrollereket használnak
- Egy adott típusra megírt programkódnak működnie kellene egy másik típuson is
- Olyan szoftverkörnyezetet kell kialakítani, amelyben különböző gyártó eszközei egységesen kezelhetők



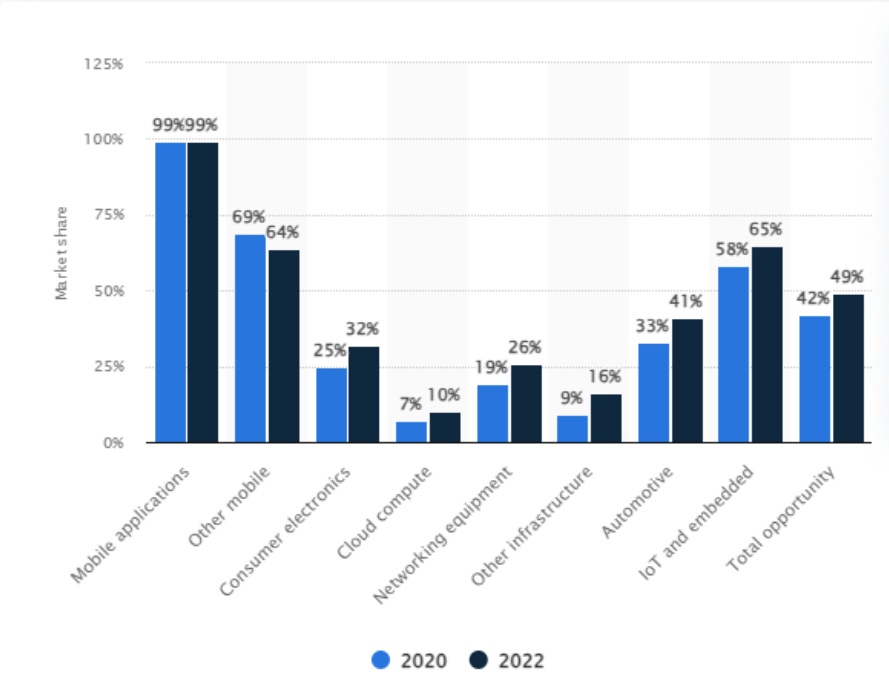
CMSIS

- CMSIS: Common Microcontroller Software Interface Standard
- Szabványosítási törekvés.
- Az ARM-Cortex-M processzorok támogatásából nőtt ki
  - Később Cortex-A típusokhoz is használták
  - Kvázi ipari szabványnak nőtte ki magát
- Szoftverkomponensek és perifériák szabványosított használatát tűzi ki célul.
  - Ezáltal többféle uC és fordító használatához nyújt szabványos lehetőséget

ARM mikrovezérlők

- ARM (Advanced RISC Machine)
- IP (Intellectual Property)-ként árulja processzormagjait különböző gyártóknak: TI, NXP, ST, … szinte minden gyártó

Arm Holdings market share across key technology markets worldwide in 2020 and 2022

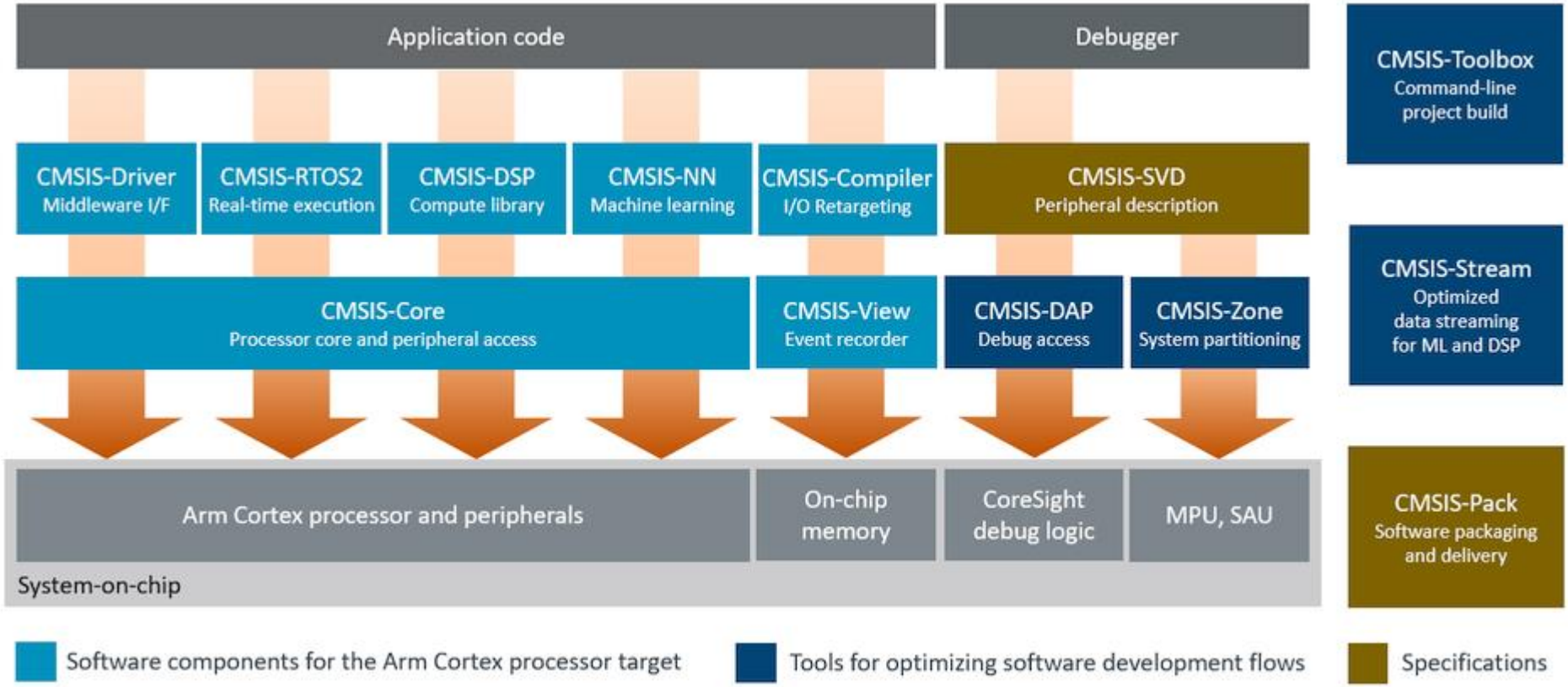


CMSIS előnyei

- A CMSIS csökkenti:
  - tanulási görbét,
  - fejlesztési költségeket,
  - piacra kerülési időt.
- A fejlesztők a könnyen használható, szabványosított szoftverfelületeken keresztül gyorsabban írhatnak szoftvert.
- Az egységes szoftverfelületek javítják a szoftver hordozhatóságát és újrafelhasználhatóságát.
- Az általános szoftverkönyvtárak és interfészek konzisztens szoftverkeretet biztosítanak.
- A program hibakeresést a hibakeresők számára perifériainformációkkal és a printf stílusú kimenethez ITM-csatornákkal javítja.
- Mivel fordítótól független réteg, lehetővé teszi többféle fordítóprogram használatát. (főbb fordítók támogatják.)
- A CMSIS-t CMSIS-Pack formátumban teszik közzé, ami lehetővé teszi a szoftver gyors alkalmazását, egyszerűsíti a frissítéseket, és konzisztens integrációt tesz lehetővé a fejlesztőeszközökbe.

CMSIS felépítése

- Réteges szerkezet:



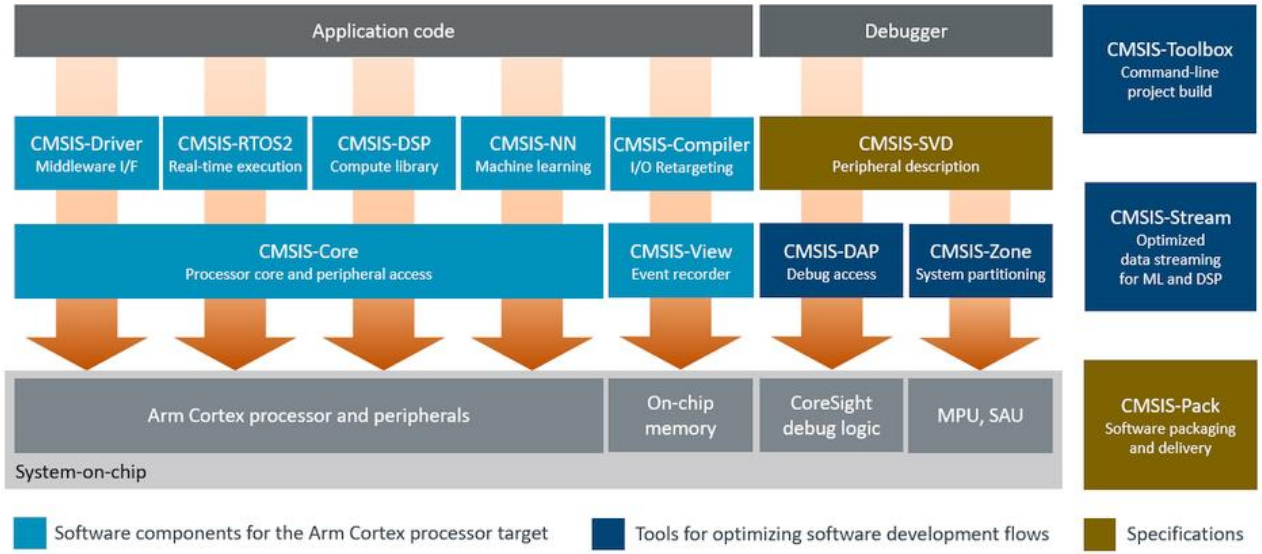
Forrás: https://arm-software.github.io/CMSIS_6/latest/General/index.html

CMSIS - fájlszerkezet

- Összehasonlítva a fájlszerkezetet a specifikációval:

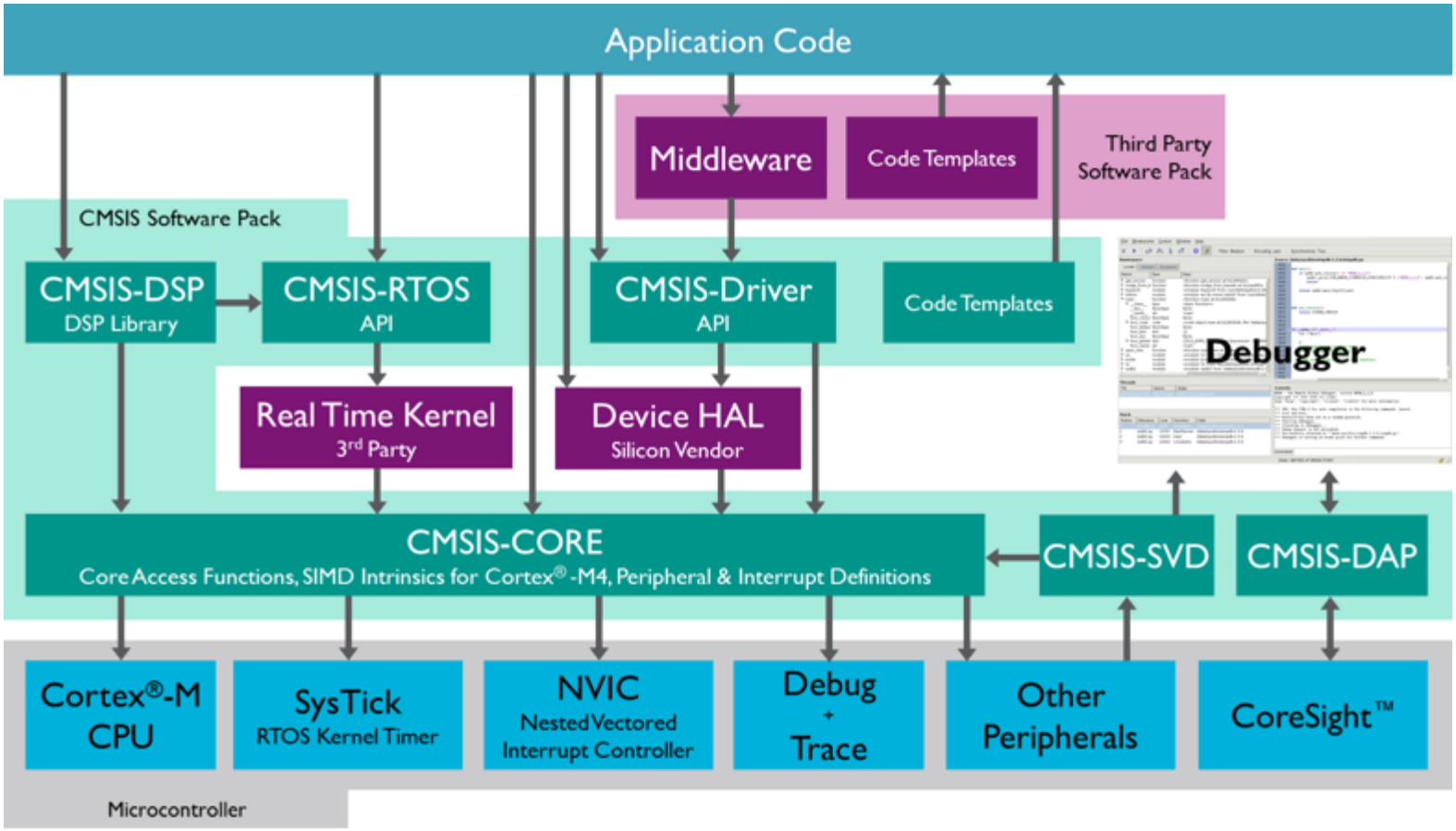
c:\Users\orosz\SimplicityStudio\SDKs\gecko_sdk\platform\CMSIS*.*

Name	Ext	Size
[..]		<DIR>
[component]		<DIR>
[Core]		<DIR>
[Core_A]		<DIR>
[DAP]		
[Driver]		
[NN]		
[Pack]		
[RTOS]		
[RTOS2]		
[template]		
[Utilities]		
LICENSE		



CMSIS felépítése

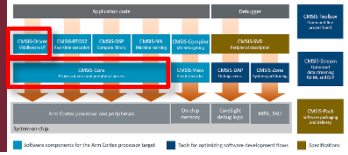
- Részletesebb struktúra



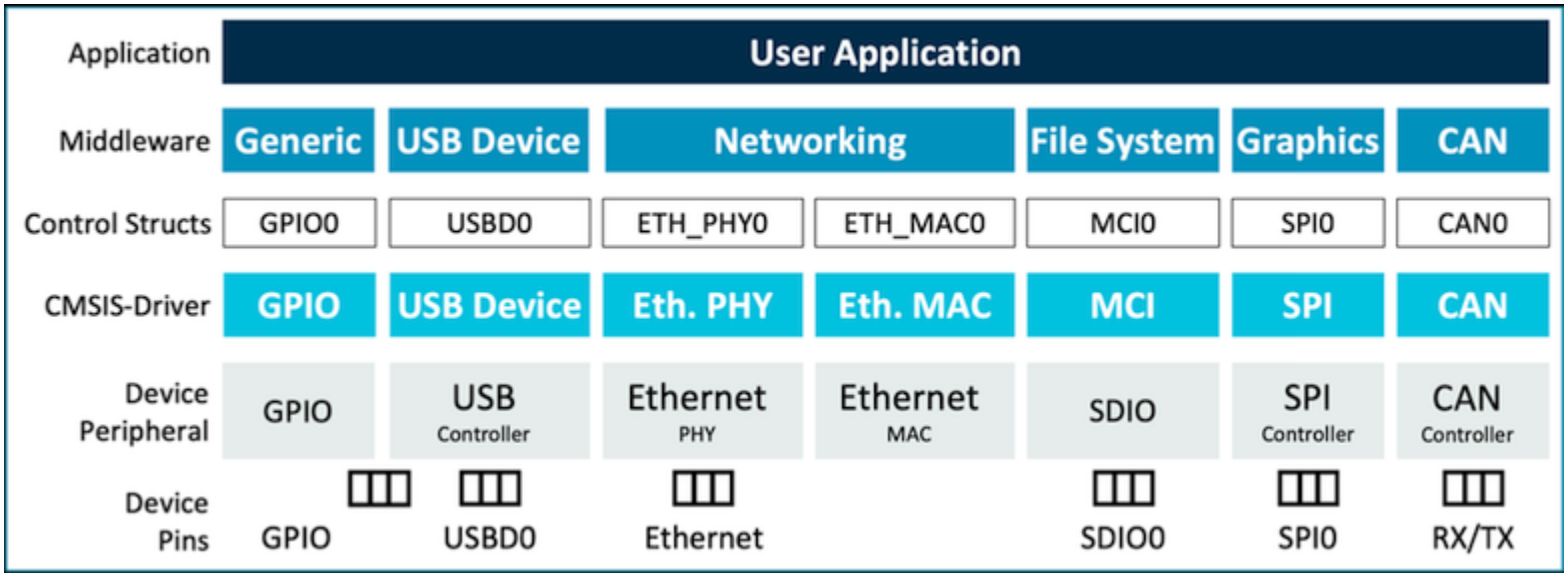
CMSIS felépítése

- Réteges szoftverstruktúra a kompatibilitás megőrzéséhez
- Definiálva van, hogy milyen rétegeket szolgáltat a processzormag tervezője, a mikrokontroller gyártója, egyéb szoftverkomponensek gyártója és az alkalmazás fejlesztője

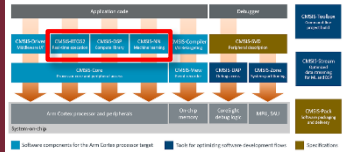
CMSIS komponensek



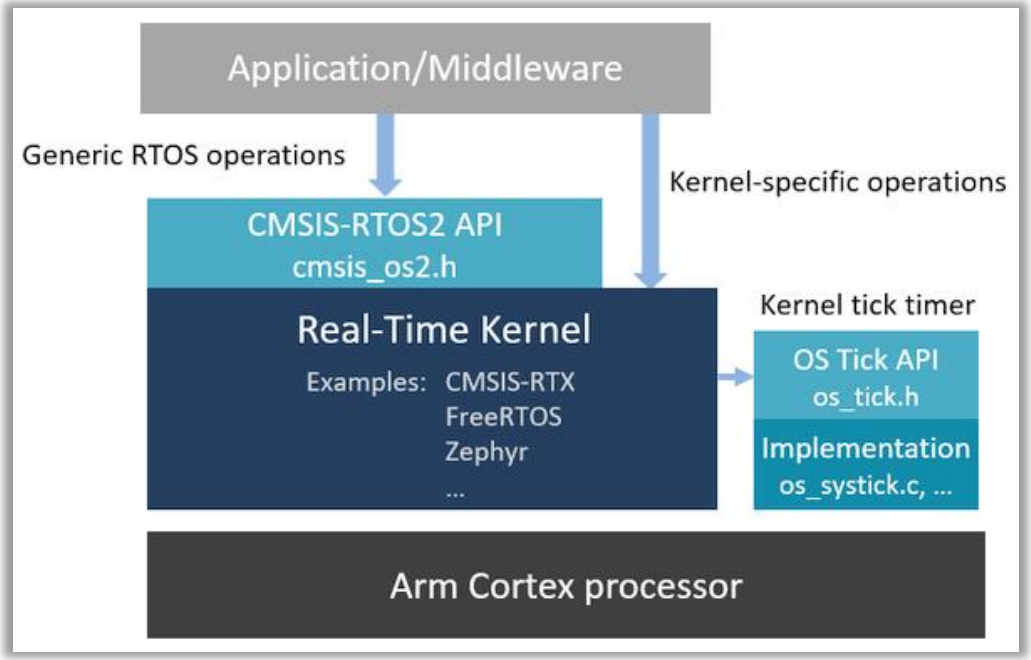
- Core: API és futtatókörnyezet a Cortex-M és A típusú mikrokontrollerek processzormagjához és processzor *belső* perifériáihoz.
- Driver: a perifériákhoz nyújt hozzáférést a middleware szoftverrétegek számára.
  - Middleware: alkalmazói és driver közötti réteg, pl. kommunikációs alapfunkciók megvalósítása, fájlkezelő rendszer megvalósítása



CMSIS komponensek



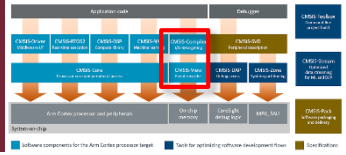
- RTOS: valós idejű operációs rendszerek támogatása, pl.:



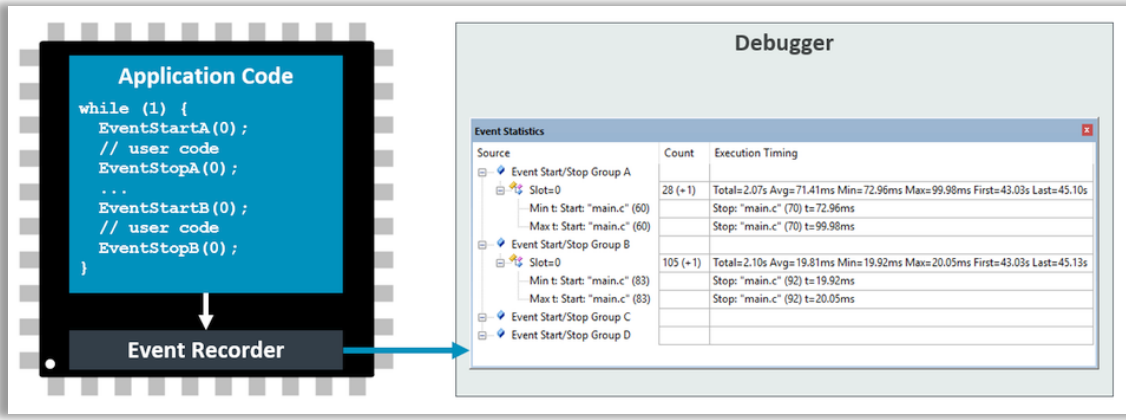
  * időzítés
  * memóriamenedzsment
  * folyamatok izolálása

- DSP&NN: jelfeldolgozási műveletek és neurális hálózatok hatékony megvalósítását támogató szoftverkomponensek. (fix- és lebegő pontos számítások, SIMD mód, különböző matematikai függvények megvalósítása)

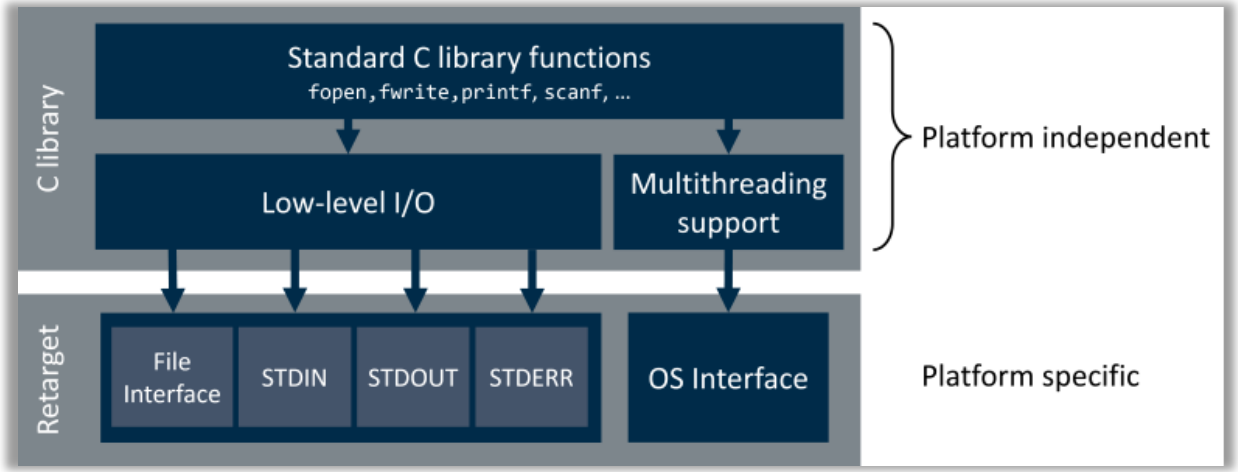
CMSIS komponensek



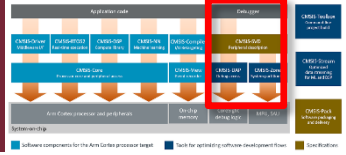
- View: debugolást segítő komponens
  - Event recorder: események rögzítése
  - Események: szűrése, megjelenítése



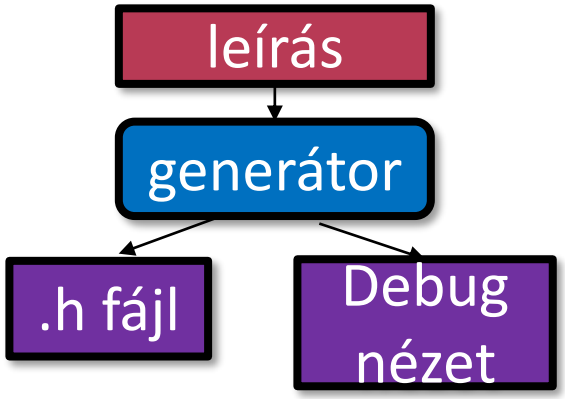
- Compiler:
  IO műveletek C könyvtári függvényekhez



CMSIS komponensek

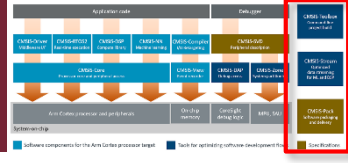


- SVD (System View Description): perifériák szabványosított leírása
  - Összekapcsolja a header fileokat és debug információkat
    - (lásd: 1. gyakorlat, GPIO regiszter vizsgálata)
  - Periférialeírást egységesíti (regiszterek, bitmezők, IT-k)
  - Újabb verzió esetén egyből alkalmazható
  - Automatizálhatóan generálhatók az egyes fájlok a leírásból

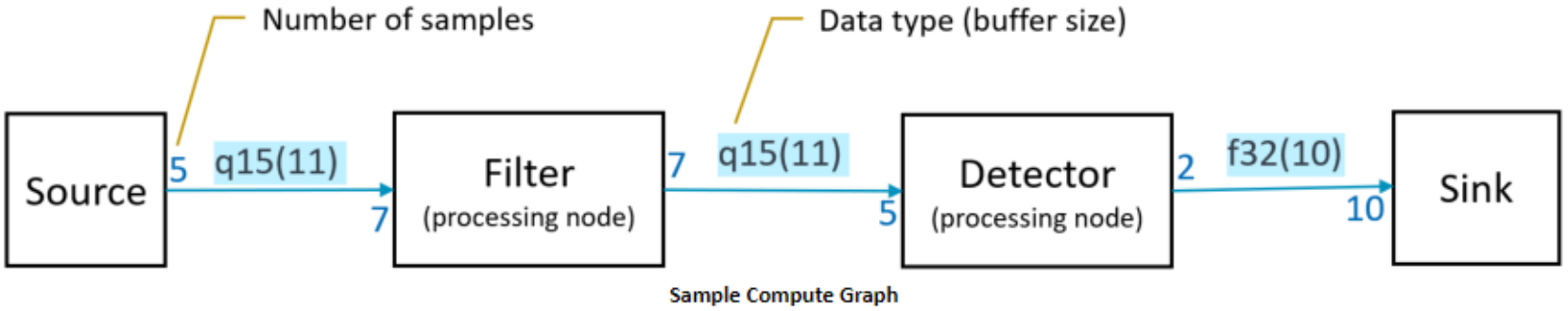


- DAP (Debug Access Port): egységes interfészt biztosít a debuggolást segítő uC periféria eléréséhez
- Zone: különböző privilégiummal rendelkező szoftverkomponensek kezelését segíti (pl. OS esetén alkalmazási szintű és kernel szintű feladatok)

CMSIS komponensek

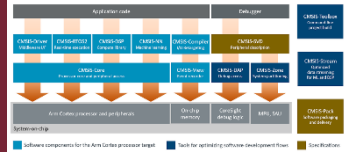


- Pack/Toolbox: egy egységes szoftverstruktúrát definiál, és sztenderdizált fejlesztési folyamatot függetlenül a fordítótól.
- Stream: adatfeldolgozó alkalmazásokhoz egy segédeszköz, amely egy adatfolyamdiagram alapján képes a feldolgozás optimalizálására



Sample Compute Graph

CMSIS - CORE



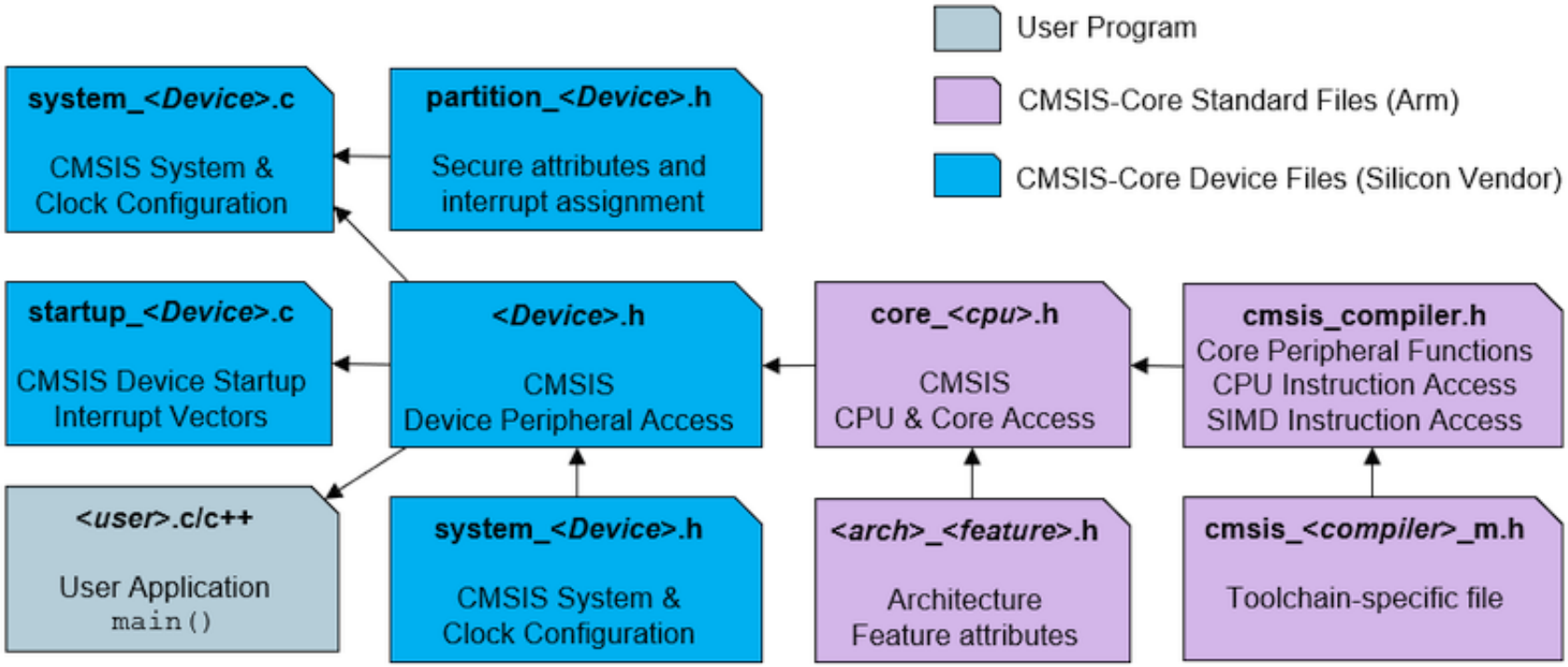
- A CMSIS-Core (Cortex-M) komponens az Arm Cortex-M eszközök hozzáférést biztosít a felhasználó számára a processzormaghoz és az eszköz perifériáihoz:
  - **Hardware Abstraction Layer (HAL):** Az összes Cortex-M variánshoz egy standardizált periféria- és regiszterkészlet-kezelés a SysTick, NVIC, System Control Block, MPU, FPU regiszterszterekhez
  - **Rendszerkivétel nevek:** A rendszerkivételekhez, interruptokhoz való hozzáférés definiálása
  - **Header file szervezés definíciók:** Elnevezési konvenciók a mikrokontroller-specifikus interruptokhoz és header file-okhoz
  - **Rendszerindítás:** Mikrovezérlő-független inicializáló függvény interfész. Standardizált SystemInit() függvény
  - **Speciális utasítások támogatása**
  - Globális változó a **system clock** frekvencia meghatározására

CMSIS – CORE nevezéktan

- Jelölések
  - **NAGYBETŰ** írásmód**:** Core és periféria regiszter, CPU utasítások
  - **CamelCase** írásmód: a függvény és megszakítások elnevezése.
  - **Namespace_** előtag: a névtér elnevezés, hogy azonosíthatók legyenek a perifériák, könyvtárak és alkalmazói programok hasonló funkciójú elemei (GPIO_PinModeGet, TIMER_Init, UART_Init).

CMSIS - CORE

- CMSIS-Core standard fájlok: ARM szolgáltatja
- CMSIS-Core device fájlok: mikrokontroller-gyártó szolgáltatja

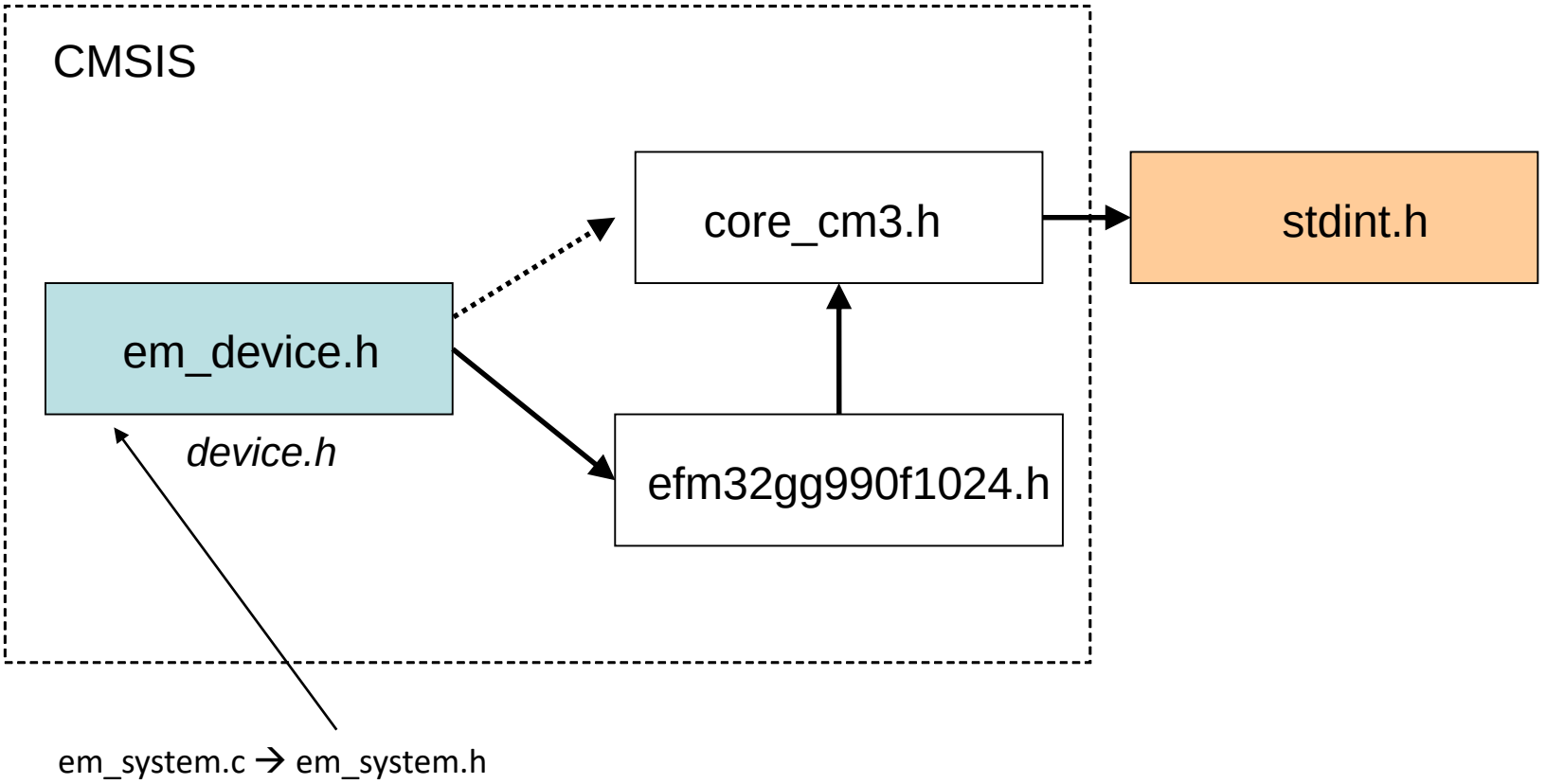


A CMSIS file-jai és könyvtárstruktúrája

File	Szolgáltató	Leírás
device.h	Mikrovezérlőgyártó	A mikrovezérlő perifériáinak definíciója. Ez a file include-olhatja az összes többi mikrovezérlőhöz tartozó firmware
startup *device*	ARM, de a mikrovezérlő, vagy compiler gyártó adoptálhatja és testreszabhatja	A Cortex-M mikrovezérlő startup kódja és a teljes mikrovezérlőfüggő ugrótábla
system_device	ARM, de a mikrovezérlő gyártó, adoptálhatja és testre szabhatja	Mikrovezérlőfüggő inicializációs rész. Tipikusan a PLL és egyéb órajelforrások inicializálása történik itt.
core_cm(x).h	ARM (fordítófüggő részekkel)	A Cortex-M(x) processzor alapperiférіáinak és regisztereinek
core_cmFunc.h	ARM (fordítófüggő részekkel)	A Core regisztereihez való hozzáférés (státusz regiszter, IT priroritás stb.). Sokszor egy külön fordítófüggő header file
core_cmInstr.h	ARM (fordítófüggő részekkel)	A Core átlal támogatott utasításokhoz nyújt C nyelvű hozzáférést. Sokszor egy külön fordítófüggő header file
core_cmSimd.h	ARM (fordítófüggő részekkel)	A Core átlal támogatott SIMD utasításokhoz nyújt C nyelvű hozzáférést. Sokszor egy külön fordítófüggő header file
core_math.h	ARM (fordítófüggő részekkel)	Speciális DSP függvényekhez ad hozzáférést, Inline függvények, vagy egy előre letöltött library-ra hivatkoznak

A device.h szerepe

- Az egyetlen szükséges include file a felhasználó számára (az induláshoz)



Regisztertömbök strukturált kezelése

- 1. lépés: a regisztertömböknek megfelelő struktúra létrehozása
  - Példa: GPIO-hoz tartozó regiszterkészlet (C kód + adatlap):
    - efm32gg990f1024.h→efm32gg_gpio.h→GPIO_TypeDef→GPIO_P_TypeDef

```
typedef struct {
  __IOM uint32_t CTRL;
  __IOM uint32_t MODEL;
  __IOM uint32_t MODEH;
  __IOM uint32_t DOUT;
  __OM uint32_t  DOUTSET;
  __OM uint32_t  DOUTCLR;
  __OM uint32_t  DOUTTGL;
  __IM uint32_t  DIN;
  __IOM uint32_t PINLOCKN;
} GPIO_P_TypeDef;
```

Offset	Name
0x000	GPIO_PA_CTRL
0x004	GPIO_PA_MODEL
0x008	GPIO_PA_MODEH
0x00C	GPIO_PA_DOUT
0x010	GPIO_PA_DOUTSET
0x014	GPIO_PA_DOUTCLR
0x018	GPIO_PA_DOUTTGL
0x01C	GPIO_PA_DIN
0x020	GPIO_PA_PINLOCKN

Regisztertömbök strukturált kezelése

- Regisztertípus-definíciók: core_cm3.h
- Általános típusdefiníciók regiszterek eléréséhez (fordítófüggetlen)

```
/* following defines should be used for structure members */
#define   __IM    volatile const    /*! Defines 'read only' structure member permissions */
#define   __OM    volatile          /*! Defines 'write only' structure member permissions */
#define   __IOM   volatile          /*! Defines 'read / write' structure member permissions */
```

Fontos, hogy volatile változók legyenek, különben a fordító kiveheti a nem használt mezőket, így elcsúszna a teljes struktúra

- Portleíró struktúrák tömbösítése (ahány port van):

```
typedef struct {
  GPIO_P_TypeDef P[6U];          /**< Port configuration bits */
  uint32_t       RESERVED0[10U]; /**< Reserved for future use **/
```

Regisztertömbök strukturált kezelése

- 2. lépés: adott perifériához tartozó regisztertömb báziscímének kikeresése (efm32gg990f1024.h):



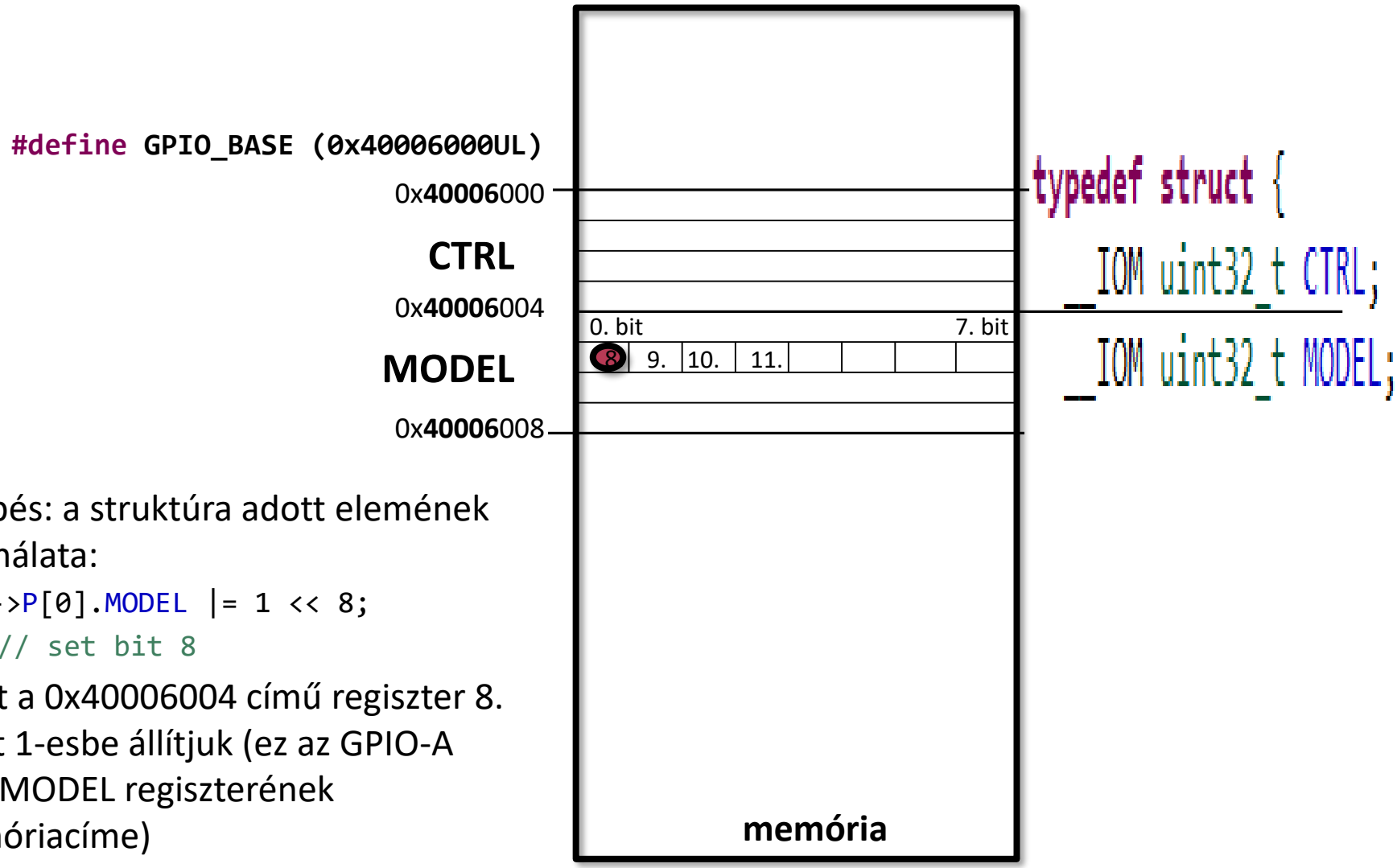

```
#define GPIO_BASE   (0x40006000UL) /**< GPIO base address  */
```

- 3. lépés: egy adott típusú struktúrára mutató pointert ráállítani a megfelelő memóriacímre (efm32gg990f1024.h):

```
#define GPIO   ((GPIO_TypeDef *) GPIO_BASE)  /**< GPIO base pointer */
```

- 4. lépés: a struktúra adott elemének használata:
  - `GPIO->P[0].MODEL |= 1 << 8;  // set bit 8`
  - Tehát a 0x**60006**004 című regiszter 12. bitjét 1-esbe állítjuk (ez az ADC command regiszterének memóriacíme, másként a 0x40006004 című regiszter 4. bitje)

Regisztertömbök strukturált kezelése



- 4. lépés: a struktúra adott elemének használata:
- `GPIO->P[0].MODEL |= 1 << 8;`
  `// set bit 8`
- Tehát a 0x40006004 című regiszter 8. bitjét 1-esbe állítjuk (ez az GPIO-A port MODEL regiszterének memóriacíme)
- A regiszterek 32 bitesek (4 bájtosak)

Startup kód

- Funkciók:
  - Megszakítási vektortábla tárolása
  - Változók inicializálása (RAM feltöltése a flash-ből)
  - Rendszer inicializálása
  - Belépési pont a főprogramhoz
- Mintapélda: project → gecko_sdk_xxx → platform → Device → SiliconLabs → EFM32GG → Source → startup_efm32gg.c



Startup kód

- Startup kód főbb részei:
  - Interrupt vektorok definiálása
    - Interrupt handlerek (lásd későbbi előadások)
    - Vektortábla definiálása megszakításnevekkel
    - weak függvények: felüldefiniálható (példát lásd később: IT)

```
/*------------------------------------------------------------------------
 * Exception / Interrupt Handler
 *------------------------------------------------------------------------*/
/* Cortex-M Processor Exceptions */
void NMI_Handler(void)              __attribute__ ((weak, alias("Default_Handler")));
void HardFault_Handler(void)        __attribute__ ((weak, alias("Default_Handler")));
void MemManage_Handler(void)        __attribute__ ((weak, alias("Default_Handler")));
void BusFault_Handler(void)         __attribute__ ((weak, alias("Default_Handler")));
void UsageFault_Handler(void)       __attribute__ ((weak, alias("Default_Handler")));
void DebugMon_Handler(void)         __attribute__ ((weak, alias("Default_Handler")));
void SVC_Handler(void)              __attribute__ ((weak, alias("Default_Handler")));
void PendSV_Handler(void)           __attribute__ ((weak, alias("Default_Handler")));
void SysTick_Handler(void)          __attribute__ ((weak, alias("Default_Handler")));
void sl_app_properties(void);       /* Prototype to please MISRA checkers. */
void sl_app_properties(void)        __attribute__ ((weak, alias("Default_Handler")));

/* Part Specific Interrupts */
void DMA_IRQHandler(void)           __attribute__ ((weak, alias("Default_Handler")));
void GPIO_EVEN_IRQHandler(void)     __attribute__ ((weak, alias("Default_Handler")));
```

```
/*------------------------ *
Default Handler for Exceptions
/ Interrupts
 *------------------------*/
void Default_Handler(void)
{
  while (true) {
  }
}
```

Startup kód

- Startup kód főbb részei:
  - Interrupt vektorok definiálása
  - Rendszerinicializálás
    - SystemInit(): rendszer alapvető inicializálása
  - Változók inicializálása:
    - __PROGRAM_START() → __cmsis_start()

```
/*----------------------------------------------------------------------------
 * Reset Handler called on controller reset
 *----------------------------------------------------------------------------*/
__NO_RETURN void Reset_Handler(void)
{
#ifndef __NO_SYSTEM_INIT
  SystemInit();                 /* CMSIS System Initialization */
#endif

#if defined (__GNUC__) && defined (__START)
  Copy_Table();
  Zero_Table();
  __START();
#else
  __PROGRAM_START();            /* Enter PreMain (C library entry point) */
#endif /* __GNUC__ */
}
```

Startup kód

- Változók inicializálása és C program belépési pontja

```
__STATIC_FORCEINLINE __NO_RETURN void __cmsis_start(void)
{
  extern void _start(void) __NO_RETURN;

  typedef struct {
    uint32_t const* src;
    uint32_t* dest;
    uint32_t  wlen;
  } __copy_table_t;

  typedef struct {
    uint32_t* dest;
    uint32_t  wlen;
  } __zero_table_t;

  extern const __copy_table_t __copy_table_start__;
  extern const __copy_table_t __copy_table_end__;
  extern const __zero_table_t __zero_table_start__;
  extern const __zero_table_t __zero_table_end__;

  for (__copy_table_t const* pTable = &__copy_table_start__; pTable < &__copy_table_end__; ++pTable) {
    for(uint32_t i=0u; i<pTable->wlen; ++i) {
      pTable->dest[i] = pTable->src[i];
    }
  }

  for (__zero_table_t const* pTable = &__zero_table_start__; pTable < &__zero_table_end__; ++pTable) {
    for(uint32_t i=0u; i<pTable->wlen; ++i) {
      pTable->dest[i] = 0u;
    }
  }

  _start();
}
```

Inicializált változók feltöltése: .data szegmens

Nem inicializált változók nullázása: .bss szegmens

Főprogram belépési pontja

system_device.c

- Példa: gecko_sdk_4.4.3 → platform → Device → SiliconLabs → EFM32GG → Source → system_efm32gg.c
  - void SystemInit(void):
    - rendszer inicializálása, pl. interrupt vektortábla beállítása
    - Startup kódból hívódik meg, de lehet, hogy a felhasználónak kell meghívnia
  - Órajelkezelő függvények:
  - `uint32_t SystemCoreClock`
    - Rendszerórajel frekvenciáját tartalmazza

```
static __INLINE void SystemCoreClockUpdate(void)
{
  (void)SystemCoreClockGet();
}

uint32_t SystemCoreClockGet(void);
uint32_t SystemMaxCoreClockGet(void);
void SystemInit(void);
uint32_t SystemHFClockGet(void);
uint32_t SystemHFXOClockGet(void);
void SystemHFXOClockSet(uint32_t freq);
uint32_t SystemLFRCOClockGet(void);
uint32_t SystemULFRCOClockGet(void);
uint32_t SystemLFXOClockGet(void);
void SystemLFXOClockSet(uint32_t freq);
```

Fordítóspecifikus definíciók

- Vannak fordítóspecifikus attribútumok, amiknek standardizált neveket rendelünk (platform → CMSIS → Core → Include → cmsis_gcc.h):

```
/* CMSIS compiler specific defines */
#ifndef   __ASM
  #define __ASM                    __asm
#endif
#ifndef   __INLINE
  #define __INLINE                 inline
#endif
#ifndef   __STATIC_INLINE
  #define __STATIC_INLINE          static inline
#endif
#ifndef   __STATIC_FORCEINLINE
  #define __STATIC_FORCEINLINE
__attribute__((always_inline))   static inline
#endif
#ifndef   __USED
  #define __USED                   __attribute__((used))
#endif
#ifndef   __WEAK
  #define __WEAK                   __attribute__((weak))
```

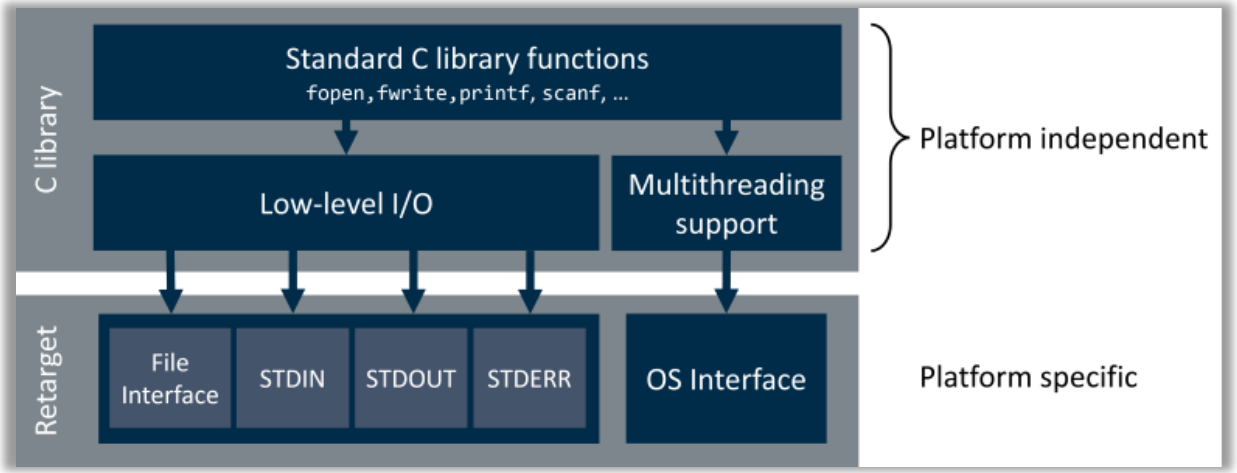
Speciális utasítások támogatása

- A mikrokontrollerek tartalmazhatnak speciális gépi kódú utasításokat, amelyeket a standard C nyelv nem támogat.
- Ezen utasítások használatát egységesíti, pl. (platform → CMSIS → Core → Include → cmsis_gcc.h):

```
#define __NOP()                                 __ASM volatile ("nop")

/**
  \brief   Wait For Interrupt
*/
#define __WFI()                                 __ASM volatile ("wfi":::"memory")
/**
  \brief   Enable IRQ Interrupts
*/
__STATIC_FORCEINLINE void __enable_irq(void)
{
  __ASM volatile ("cpsie i" : : : "memory");
}
```

Standard C függvények beágyazott rendszerekben



- Néhány, standard C függvény beágyazott rendszerekben történő felhasználása kérdéses lehet: printf, scanf…
  - Hova íratjuk ki? Milyen fájlt használunk?
- Függvénykönyvtár: newlib
  - Támogatja a platformfüggetlen utasítások megvalósítását a platformspecifikus részek implementálásával

Standard C függvények beágyazott rendszerekben

- A szabvány definiál alacsony szintű függvényeket, amelyek segítségével megvalósíthatók komplex funkciók
  - Példa: CMSIS_example / gecko_sdk_4.4.3 / platform / common / src / sl_syscalls.c
  - _exit: a main függvényből kilépve milyen állapotba kerüljön a processzor
  - _write: ezen függvény segítségével írja ki a printf az összeállított stringet
    - Felüldefiniálva bármilyen perifériára kiírhatjuk a printf eredményét (lásd későbbi példa)
    - weak függvény: egyszer felüldefiniálható

```
__WEAK void _exit(int status)
{
  (void)status;

  /* Convince GCC that this
function never returns. */
  for (;; ) {
    ;
  }
}

__WEAK int _write(int file, const
char *ptr, int len)
{
  (void)file;
  (void)(const void *)ptr;
  (void)len;
  return 0;
}
```

Standard C függvények beágyazott rendszerekben

- A szabvány definiál alacsony szintű függvényeket, amelyek segítségével megvalósíthatók komplex funkciók
  - Példa: CMSIS_example / gecko_sdk_4.4.3 / platform / common / toolchain / src / sl_memory.c
  - _sbrk: a malloc függvény használja a memóriában történő lépkedéshez.
    - __HeapBase, __HeapLimit lásd: linkerfile.ld

```
.heap (COPY):
{
  __HeapBase = .;
  __end__ = .;
  end = __end__;
  _end = __end__;
  KEEP(*(.heap*))
  __HeapLimit = ORIGIN(RAM) + LENGTH(RAM);
} > RAM

__heap_size = __HeapLimit - __HeapBase;
__ram_end__ = 0x20000000 + 0x20000;
```

```
__USED void * _sbrk(int incr)
{
  static char *heap_end = __HeapBase;
  char *prev_heap_end;

  if ((heap_end + incr) > __HeapLimit) {
    // Not enough heap
    return (void *) -1;
  }

  prev_heap_end = heap_end;
  heap_end += incr;

  return prev_heap_end;
}
```

https://community.silabs.com/s/article/malloc-and-sbrk?language=ko

Változók formázott kiíratása

- A printf függvény PC-n a sztenderd kimenetre (monitor) írja a szöveget, de beágyazott rendszereknél nincsen ilyen általános sztenderd kimenet
- Kényelmes: formázott szöveget készít, jól lekezeli a típusokat (pl. float számok kiíratása)
- A kimenetet át kell irányítani valamilyen perifériára
  - UART
  - Kijelző
- Átirányítás:
  - Fordítófüggő, több módszer is lehet
  - Példák:
    - Simplicity Studio: felüldefiniáljuk a _write függvényt. Ez egy „gyenge” (weak) függvény, amit egy új definíció felülír. A printf függvény ezt a függvényt hívja meg kiíratáskor

```
int _write(int file, char *data, int len) {
    SegmentLCD_Write(data);
    return len;
}
```

Változók formázott kiíratása

- Vigyázat, a printf függvény erőforrás-igényes!
- Példa: kiíratás az LCD-re
  - Kiíratás a normál LCD kezelő függvénnyel.
  - Programmemória igénye összesen: 11272 byte

```
SegmentLCD_Init(false);
SegmentLCD_Write("Pelda");
```

```
Running size tool
arm-none-eabi-size "print_proba.axf"
   text    data     bss     dec     hex filename
  11272     124      68   11464    2cc8 print_proba.axf
```

  - Kiíratás a printf függvény segítségével úgy, hogy átirányítjuk a kimenetet az LCD-re
  - Programmemória igénye összesen: 14924 byte
  - Különbség: 3652 byte, ez egy kisebb mikrokontrollert majdnem teljesen lefoglalna!!!

```
SegmentLCD_Init(false);
//SegmentLCD_Write("LCD");
printf("Print \n");
```

```
int _write(int file, char *data, int len) {
    SegmentLCD_Write(data);
    return len;
}
```

```
Running size tool
arm-none-eabi-size "print_proba.axf"
   text    data     bss     dec     hex filename
  14924     124      84   15132    3b1c print_proba.axf
```

  - Különösen formátumkonverzió esetén a futási idő sem elhanyagolható.

Perifériák kezelése

- Perifériák kezelésére szabványos felépítés: Init, ReInit és perifériát kezelő függvények, példa:
- https://arm-software.github.io/CMSIS_6/latest/Driver/group__usart__interface__gr.html
- Inicializáláshoz perifériánként változó számú és jellegű paraméter
  - a paraméterek lehetnének függvényparaméterek
  - Ez rosszul kezelhető, paraméterek sorrendjét könnyű elrontani
  - Struktúra elemeiként adjuk át a periféria beállítási paramétereit
- Példa: UART inicializálása

```
uart0_init.baudrate = 115200;
uart0_init.databits = usartDatabits8;
uart0_init.enable = usartEnable;
uart0_init.stopbits = usartStopbits1;

USART_InitAsync(UART0, &uart0_init);
```

A CMSIS coding guideline-jai

- A CMSIS szabvány a MISRA 2004-es guideline-jainak betartásával készül.
- A CMSIS az adattípusokként az **<stdint.h>** -ba definiált típusokat használja.
  - Pl.: int lehet 16 vagy 32 bites architektúrától függően
- A kifejezéseket tartalmazó #define-okat zárójelezni kell.
- A *Core Peripheral Access Layer (**CPAL**)* összes függvénye újrahívható (re-entrant) kell, hogy legyen.
- A *Core Peripheral Access Layer (**CPAL**)* nem tartalmazhat blokkoló kódot.
- Az összes interrupt-kezelő függvények az **_IRQHandler** utótaggal kell záródnia.

Szoftverimplementálás általános szabályai

- Szinte SIL/ASIL szint függetlenül mindig kötelezőek

	Topics	ASIL			
		A	B	C	D
1a	Enforcement of low complexity	++	++	++	++
1b	Use of language subsets[b]	++	++	++	++
1c	Enforcement of strong typing[c]	++	++	++	++
1d	Use of defensive implementation techniques	o	+	++	++
1e	Use of established design principles	+	+	+	++
1f	Use of unambiguous graphical representation	+	++	++	++
1g	Use of style guides	+	++	++	++
1h	Use of naming conventions	++	++	++	++

Style guides

- Cél a kód egységes kinézete és szervezése
  - Nem egységes kód nehezíti a csapatmunkát és az átláthatóságot
- Nincs egységes nemzetközi szabvány
- Tipikusan cégekre jellemző szabályozások
  - C és header file-ok szerkezete
  - Változók elnevezésének stílusa
  - Vezérlési szerkezetek stílusa
  - Kommentezés módja

MISRA-C nyelvi készlet korlátozás

- (Motor Industry Software Reliability Association)

- MISRA-C 1998: Az első MISRA C szabályverziót 1998-ban hozták létre célja az volt, hogy javítsa az UK (United Kingdom) autóiparában használt szoftverek minőségét.
- A MISRA-C 1998 sikere nem csak az autóiparra terjedt ki, hanem széles körben alkalmazásra került a repülőgépiparban valamit az egészségügyi iparban is.
- A MISRA-C 2004: Elfogadásra került az USA-ban (SAE J2632) és Japánban is.
  - Javítja a MISRA-1998 hibáit, és pontosítja azt.
  - 121 Mandatory és 20 Advisory szabályt tartalmaz a C nyelv leszűkítésére
- MISRA-C 2012: 2013 közepén megjelent új verzió a C99-es szabványára épül

MISRA-C célja

- Cél a C szabados programozói nyelv leszűkítése úgy, hogy a lehető legmegbízhatóbb nyelvi alrendszert kapjuk.
  - Szintaktikailag helyes, de potenciális hibaforrásokra megoldásokra felhívni a figyelmet.
  - Tiltani a nem strukturált programozást eredményező szerkezeteket.
  - Tiltani a nem egyértelmű változó típus használatot.
  - Szabályozni a precendencia zárójelezéseket.

Kódolási szabályok

- Kódsort soha nem szabad kikommentezni
  - A kikommentezett sorok gondot okozhatnak az egymásba ágyazott kommentek, illetve a kódérthetőség miatt is.
- Ciklusváltozót nem lehet a ciklus belsejében módosítani.

```
flag = 1;
for (i = 0; (i<5) && (flag == 1); i++)
{
        flag = 0; /* Engedélyezett a ciklus gyors befejezése */
        i = i + 3; /* Nem Engedélyezett ciklusváltozó módosítás */
}
```

- A goto használata tiltott!
- A continue és break használata korlátozott! (nem ajánlott)
- Bitmanipuláló műveletet nem szabad signed, vagy floating típusokon végrehajtani (>>, <<, ~, &, ^)

Kódolási szabályok

- Kiválasztott compiler egész osztásokra való reakcióját ellenőrizni és dokumentálni kell.
  - Egy ISO kompatibilis fordító két ponton is mutathat eltérő viselkedést negatív egész osztás esetében:
    - (-5/3) lehet -1 ahol a maradék -2 illetve
    - (-5/3) lehet -2 ahol a maradék +1
- Változók összeadásánál szorzásánál kicsúszás az értéktartományból:

```
uint16_t u16a = 40000;
uint16_t u16b = 30000;
uint32_t u32x;

U32x = u16a + u16b; /* u32x = 70000 vagy 4464? */
```

  - Az összeadás aritmetikáját nem az eredmény tárolási típusa, hanem a compiler belső aritmetikája szabályozza (belső int típus), ezért, ha a belső aritmetika 16 bites akkor az összeadásban túlcsordulás következhet be. Ilyen jellegű hibalehetőségből van még jó pár darab.
  - Explicit típuskonverziót kell alkalmazni: `U32x=(uint32_t)u16a+(uint32_t)u16b;`

Kódolási szabályok

- A kapcsos zárójelet tegyük ki az if szerkezetek után

```
if (a>b)
    a = a – b ;
    b = a + 10;   // már nem az if –hez tartozik, de ránézésre azt gondolhatnánk
```

- Feltételes szerkezetben konstanssal való összehasonlításban előre kerüljön a konstans:
  - Legyen: **if (5==i)** ne pedig **if (i==5)**.
  - Magyarázat: ha véletlenül elrontjuk, akkor az **if (5=i)** fordítási hibát dob, míg az **if (i=5)** nem okoz semmilyen szintaktikai hibát, de mégis helytelenül működik.
- Ne legyenek lekezeletlen esetek (ha egy adott eset nem fordulhat elő, jelezzük a hibát)
- Függvények visszatérési értékét ellenőrizzük. (ha nem kell ellenőrizni, legyen void)

Kódolási szabályok

- Változó számú argumentum kerülése függvények esetén (lásd printf)
- A ciklusoknak legyen fix hosszúsága (pl. for ciklus hossza legyen konstans)
  - Hibaforrás lehet, ha a ciklushossz változót rosszul határozzuk meg
- Függvény törzsében ne legyen return, lehetőleg csak egyszer, a függvény végén. (ha a függvény „közepén” térünk vissza, lehet, hogy kihagyunk bizonyos műveleteket, pl. nem engedélyezzük visszatéréskor a megszakítást).

Tipikus C file szerkezet

1. Komment a file nevéről, rendeltetéséről, szerzőjéről, verziójáról, és verzió history-ról. (Egyes verziókövető rendszerek automatikusan generálják, módosítják a file verzióra vonatkozó kommenteket. Ilyenkor ezeket a fejlesztőnek általában tilos kézzel módosítani.)
2. Header file include-ok
3. Definiciók a következő sorrendben: Típus definíciók, Define-ok, Konstans makrók függvény makrók,
4. Globális változók: extern, nem static, static global sorrendben
5. függvények: általában hívási sorrendben, ha egy mód van rá.

Tipikus H file szerkezet

1. Komment a file nevéről rendeltetéséről, szerzőjéről, verziójáról, és verzió history-ról. A file neve sohasem lehet normál C inklúdnévvel egyező pl.: „math.h”.

2. A következő file kezdési szerkezet kötelező:

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
...     /* body of example.h file */
#endif /* EXAMPLE_H *
```

3, Header file-okban ne definiáljunk változókat ha egy mód van rá.

Elnevezési szabályok

- Egyik leghíresebb az ún. *Hungarian Notatinon*
  - Simonyi Károly vezetett be a Microsoftnál
- Az elnevezési szisztéma a magyar elnevezési logikából indul ki, ahol a vezetéknév megelőzi az „adott” keresztnevet
- Ezt próbálja a változókra is bevezetni, ahol először a típus, vagy alkalmazási kör szerepel és utána csak az „adott” név
- Két alsítulsa van a System és az Application
  - *System* esetében a változó típusa
  - Az *Application* esetében az alkalmazás típusa van belekódolva a változó nevébe.
- A jelölésrendszer sokszor kiegészül a láthatóságot jelölő előtéttel is

Példák *Hungarian notation*-ra

bBusy: boolean
cApples: count of items
dwLightYears: double word (system)
fBusy: boolean (flag)
nSize: integer (system) vagy count (application)
iSize: integer (system) vagy index (application)

g_nWheels: member of a global namespace, integer
m_nWheels: member of a structure/class, integer
s_wheels: static member of a class
_wheels: local variable

Általános dokumentációs problémák

1. Megírjuk a kódot
2. Még talán kommentezzük is
3. Nagy erőszakra megírjuk a dokumentációt

4. Módosítjuk a kódot
5. Talán a kommentet is
6. A doksit biztos nem

Inkonzisztencia:

kód – komment – dokumentáció között

Automatikus dokumentáció generálás: Doxygen

- 1997-ben jelent meg az első verzió v0.1
- A komment – dokumentáció közötti inkonzisztenciát igyekszik feloldani
- Többfajta kommentezési stílust képes kezelni
- C stilusú JAVA doc szerű kommentek

```
/**
... text ...
*/
```

- C stílusú ! kommentek

```
/*!
... text ...
*/
```

Doxygen kommentek példa

```
/*! \fn void UART1_Init(unsigned long baud_rate, void (*handler)(void));
 *  \brief An inicialisation function to redirect STDIO to UART1
 *  \param baud_rate: Baudrate in bit/sec
 *  \param handler: Callback function for receiving UART characters with IT
 *  \return nothing
*/
```

```
/** @fn void UART1_Init(unsigned long baud_rate, void (*handler)(void));
 *  @brief An inicialisation function to redirect STDIO to UART1
 *  @param baud_rate: Baudrate in bit/sec
 *  @param handler: Callback function for receiving UART characters with IT
 *  @return nothing
                                                                    */
```

Csoportosítás

- Alapvetően file szintű dokumentálás, ahhoz, hogy logiaki egység szintű dokumentációt csináljunk szükséges a csoportok megalkotása

- Szinte az összes firmware library ezt alkalmazza

```
/** @addtogroup Az én csoportom
@{
*/

.....
/**
@}
*/
```

Csoportosítás, struktúra, hivatkozás

```
/** @brief  I2C Init structure definition  */
typedef struct
{
  uint32_t I2C_ClockSpeed;          /*!< Specifies the clock frequency */
  uint16_t I2C_Mode;                /*!< Specifies the I2C mode.
This parameter can be a value of @ref I2C_mode */
} I2C_InitTypeDef;
```

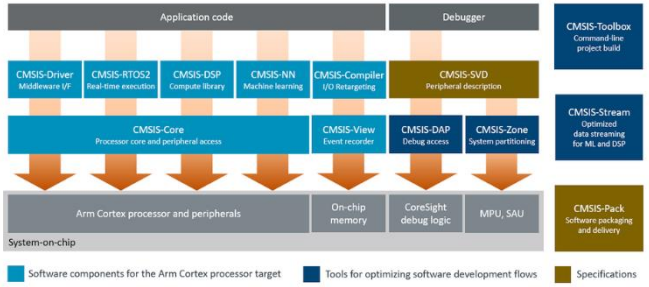
```
/** @defgroup I2C_mode

@{

*/
#define I2C_Mode_MASTER                 1
#define I2C_Mode_SLAVE                  0
/**
@}
*/
```

CMSIS összefoglaló

- Réteges szoftverarchitektúrát ír elő, jól definiált feladatokkal
- Főbb komponensek
  - Core
  - Driver, Pheripheral descriptor
  - RTOS
  - DSP, NN
  - Compiler
- Cél, hogy a hardver és a fordító minél jobban el legyen fedve az alkalmazásfejlesztő elől
  - Betanulási idő csökken
  - Kód újrahasználhatósága javul



CMSIS összefoglaló

- Ajánlás:
  - Nevezéktan
  - SW rendszer felépítése
    - IT nevek
    - Rendszerfájlok (pl. startup_efm32gg.c, device.h, core_cm3.h, cmsis_gcc.h)
      - Rendszer indulása
      - Perifériák leírása
        - » Perifériákhoz struktúradefiníciók
      - Processzor belső perifériáinak kezelése
      - Fordító interfész (fv. attribútumok, speciális utasítások, newlib,
  - API a rendszerkomponensek hozzáféréséhez
  - Perifériakezelő függvények felépítése

CMSIS összefoglaló

- Ajánlás (folyt.)
  - Kódolási konvenció
    - MISRA C alkalmazás
    - C és H fájlok szerkezete
    - Elnevezési szabályok
  - Dokumentáció (lásd doxygen)

Függvény- és változóattribútumok

https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/

https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Function-Attributes.html

https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Variable-Attributes.html

- A C nyelvben az __attribute__ ((…)) kulcsszóval lehet a függvényeknek és változóknak speciális tulajdonságokat adni. Példák (nem feltétlenül minden processzorra, és nem feltétlenül minden fordító esetén):
  - __attribute__ ((interrupt ("IRQ"))); megszakításkezelő függvényt jelöl
  - __attribute__((always_inline)): mindenképpen inline függvényként használja az adott függvényt
  - __attribute__((weak)): olyan függvényt definiál, amit máshol felül lehet definiálni. Lásd pl.: megszakításkezelés, ahol az alap megszakításkezelő függvény egy weak függvény, így ha bárhol máshol definiálok egy ugyanolyan nevű függvényt, akkor onnantól kezdve az lesz a megszakításkezelő függvény.
  - __attribute__((section("*name*"))): ha a linker fájlban megadtunk egy *name* nevű szekciót, akkor oda fog kerülni a változó.
  - __attribute__ ((__cleanup__(__iRestore))): a változó megszűnésekor meghív egy adott függvényt. AVR: ATOMIC_BLOCK, elmentjük az globális IT engedélyezésének az állapotát egy változóba, és a változó megszűnésekor visszaírjuk ezt az értéket.

Fordítási direktívák (pragma)

https://gcc.gnu.org/onlinedocs/gcc-14.2.0/gcc/Pragmas.html

- #pragma vagy _pragma: fordítási direktívák/kulcsszavak
- Akár általános, akár hardverspecifikus parancsok is adhatók, pl.:
  - #pragma once: egy adott függvényt nem include-ol többször
  - #pragma interrupt: interruptkezelő függvényt jelöl
  - #pragma align(4): mindig 4 bájt egész számú többszöröseként adódjon a kezdőcím
    - Fontos lehet pl. DSP-k esetén
  - #pragma pack: egy adott struktúra mezőit biztosan közvetlenül egymás után rendezzük
- Fordítóspecifikus, a dokumentációt meg kell nézni
- Sok hasonló funkció megvalósítható, mint az __attribute__ kulcsszóval (pl. interrupt, pack…)

Integer adattípusok használata

- A C nyelvben sok típusnál csak az van megadva, hogy legalább mekkora legyen a számábrázolási tartomány (pl. unsigned integer legalább a 0 … 65535 tartományt fedje le, de nagyobb lehet…).
  - Beágyazott rendszerek: rengeteg architektúra, lehet, hogy az int típus 16 bites, de lehet, hogy 32 bites lesz.
- Probléma: beágyazott rendszerekben nagyon sokat számít, hogy egzaktul tudjuk, hogy mi a változó bitszélessége
  - Változók leképzése pl. perifériaregiszterekre
  - Számításigény becslése
- C99 szabvány: inttypes használata
  - **#include <stdint.h>**
  - Az adott processzorra adott bitszélességű típusokat definiál, például:
    - int16_t : 16 bites előjeles egész szám (legalább 16 bit)
    - uint32_t : 32 bites előjel nélküli egész szám (pl. long unsigned int. Legalább 32 bit.)
    - Vannak minimum bitszélességek is: uint_least32_t

inline függvények

- Inline függvények: a fordító „kifejti” a függvény belsejét, és lényegében nem történik függvényhívás, hanem a függvényben található kódot helyettesíti be
  - Gyorsabb, mint a normál függvények: nincsen függvényhívási overhead.
  - Rövid, néhány parancsból álló függvények esetén éri meg.
  - Nem biztos, hogy a fordító mindig inline függvényként fogja használni, még ha annak is jelöltük.
  - static kulcsszóval szokták együtt, mert ugyanabban a fordítási egységben lehet az inline függvény.
  - Általában kivételesen a header fájlokban találhatóak (nem globális fv-ek).
- Példa:

```
uint32_t adder_fn(uint32_t x, uint32_t y) {
    return (x+y);
}
```

```
static inline __attribute__((always_inline)) uint32_t adder_fn(uint32_t x, uint32_t y){
    return (x+y);
}
```

inline függvények

- inline nélkül: 23 utasítás inline: 11 utasítás

```
117          int_num = adder_fn(x_add, y_add);
000013b8:    ldr     r3,[pc,#0x4c] ; 0x1404
000013ba:    ldr     r2,[r3]
000013bc:    ldr     r3,[pc,#0x4c] ; 0x1408
000013be:    ldr     r3,[r3]
000013c0:    mov     r0,r2
000013c2:    mov     r1,r3
000013c4:    bl      0x0000130c
000013c8:    mov     r2,r0
000013ca:    ldr     r3,[pc,#0x44] ; 0x140c
000013cc:    str     r2,[r3]
```

Függvényhívás

Visszatérés a függvényből

```
             adder_fn:
0000130c:    push    {r7}
0000130e:    sub     sp,sp,#0xc
00001310:    add     r7,sp,#0x0
00001312:    str     r0,[r7,#0x4]
00001314:    str     r1,[r7]
 75            return (x+y);
00001316:    ldr     r2,[r7,#0x4]
00001318:    ldr     r3,[r7]
0000131a:    add     r3,r2
 76          }
0000131c:    mov     r0,r3
0000131e:    adds    r7,#0xc
00001320:    mov     sp,r7
00001322:    pop.w   {r7}
00001326:    bx      lr
 83          {
```

```
117          int_num = adder_fn(x_add, y_add);
0000139e:    ldr     r3,[pc,#0x4c] ; 0x13e8
000013a0:    ldr     r2,[r3]
000013a2:    ldr     r3,[pc,#0x4c] ; 0x13ec
000013a4:    ldr     r3,[r3]
000013a6:    str     r2,[r7,#0x4]
000013a8:    str     r3,[r7]
 75            return (x+y);
000013aa:    ldr     r2,[r7,#0x4]
000013ac:    ldr     r3,[r7]
000013ae:    add     r3,r2
117          int_num = adder_fn(x_add, y_add);
000013b0:    ldr     r2,[pc,#0x40] ; 0x13f0
000013b2:    str     r3,[r2]
```

inline függvények

- Attól, hogy inline függvénynek jelölünk egy függvényt, nem biztos, hogy az fordul belőle
  - Lehet kényszeríteni, pl.: __attribute__((always_inline)) attribútummal.
  - Ha lehet, bízzuk a fordítóra, csak nagyon sebességkritikus esetekben kényszerítsük.
- Néha a fordító is észreveszi, hogy valami lehet inline függvény.

define

- Speciális szimbólumok: # és ##
- # szimbólum: stringként helyettesíti be a megadott karaktersorozatot (stringizing operator)
- ## összefűz két karaktersorozatot (Token-Pasting / merging Operator)
  - #define set(var, num, value) var##num = #value
  - set(def_var, 3, 2)
  - Mivé fordul?
  - def_var3 = "2";
- Vigyázni kell, mert átláthatatlan kódot eredményezhet.

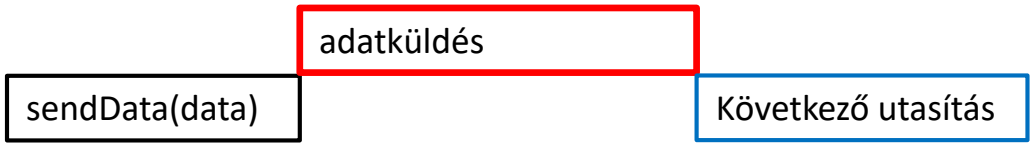
idiom recognition

- idiom recognition (idiom fordítása: szólás, kifejezés)
  - Adott parancsképet felismer a fordító, és az adott processzor speciális utasításaira képes lefordítani
  - Példák (fordítótól is függ):
    - Telítés (Cortex SSAT asm parancs): Y = (x<-8)? -8 : (x>7 ? 7:x)
    - Cirkuláris buffer (DSP): a+=w[j]*x[i % N]
      - A moduló osztást valójában nem végzi el, hanem a hardveres cirkuláris buffert használja.
  - Nem kell speciális függvényeket használni, így más processzorra is lefordul a program, viszont adott processzor esetén hatékony kódot tud generálni a fordító.
  - Nem biztos, hogy minden fordító felismeri (még adott processzora sem biztos).
  - Ismernie kell a programozónak, hogy mik a lehetőségek.
  - FPGA-k esetén is fontos, hogy általánosan elterjedt leírást használjunk, egyébként a szintézer nem biztos, hogy észreveszi, mit szeretnénk.

Könyvtári függvények használata

- Tisztában kell lenni, hogy egy adott függvény:
  - Milyen szinten használja ki a perifériákat,
  - Milyen erőforrást igényel,
  - Igényel-e inicializálást (pl. adatküldés előtt inicializálni kell-e az adott perifériát).
- Blokkoló/nem blokkoló függvény
  - A futás befejezte előtt visszatér-e a függvény.
  - Például üzenetküldés soros porton keresztül:
    - a teljes adatcsomag elküldésekor tér vissza a függvény,
    - vagy átveszi a teljes tömböt, és a háttérben elküldi, a futás joga pedig visszakerül a főprogramhoz.

Blokkoló adatküldés: addig nem tér vissza a függvény, amíg minden adatot el nem küldött:



Nemblokkoló adatküldés: a küldés előkészítése után visszatér a függvény és háttérben megy az adatküldés:

